

Politecnico di Milano

Anno accademico 2010-2011

Ingegneria del Software

Appello del 9 Febbraio 2012

Cognome:	
Nome:	Matricola:

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 1h45m.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 1

Si scrivano in JML le pre- e post- condizioni dei seguenti metodi:

- **cornici** prende in ingresso una matrice bidimensionale quadrata di interi e controlla che i numeri di ogni cornice siano tutti uguali tra loro. Ad esempio, la matrice a sinistra di Figura 1 non rispetta la condizione, mentre quella di destra è corretta. Si noti inoltre che la matrice con un solo elemento rispetta sempre la condizione e che lo stesso vale per tutte le matrici quadrate i cui elementi sono tutti uguali tra loro.

1	1	1
2	2	2
3	3	3

1	1	1	1	1
1	3	3	3	1
1	3	2	3	1
1	3	3	3	1
1	1	1	1	1

Figure 1: Matrici d'esempio.

public static boolean cornici(int[][] a)

- **somme** prende in ingresso un array di interi, di lunghezza n, e controlla che ogni elemento, di posizione compresa tra 0 e $n - 3$, sia la somma dei due elementi successivi.

public static boolean somme(int[] a)

Esercizio 2

Un Dizionario è una struttura dati adatta a memorizzare e ricercare efficientemente degli elementi di un tipo assegnato. Ogni elemento contenuto è caratterizzato da una chiave.

Si consideri la seguente specifica della classe Dizionario, generica rispetto ai parametri $< V, K >$ che rappresentano rispettivamente il tipo dei valori restituiti dal dizionario e il tipo delle chiavi. Il Dizionario utilizza una classe Elemento, anch'essa generica, di cui ogni oggetto eìndividuato da un valore e da una chiave.

```
import java.util.*;
/*@ pure @*/ class Elemento<Val,Key> {
//@ensures k == key() && v == value();
public Elemento (Key k, Val v) { };
//restituisce il valore di this;
public Val value() {return v;}
//restituisce la chiave di this;
public Key key() {return k;}
}

public class Dizionario<V,K> {
private Elemento<V,K> el;

//costruisce un DizionarioVuoto
public Dizionario() {}

//restituisce numero di elementi del dizionario
public int size() {}

//@ensures (* se esiste un elemento di chiave 'chiave' allora \result.key().equals(ch
//@ealtrimenti \result == null
public Elemento<V,K> find(K chiave){}

//restituisce una lista di tutti e soli gli elementi di this, in ordine arbitrario;
public ArrayList<Elemento<V,K>> elementi() {}

//inserisce in this e restituisce un nuovo elemento di chiave k e valore v
public Elemento<V,K> insert (K k, V v) {}

//rimuove e da this
public void remove (Elemento<V,K> e){}
}

}
```

a) Scrivere la specifica JML di insert e remove.

Sketch Soluzione

```
//@\result.key() == k && elementi().contains(\result) &&
//@(\forall int i; 0\le i && i < \old(elementi().size());
//@      \old(elementi().get(i)).key() != k ==>
//@       elementi().contains(\old(elementi().get(i)))) &&
//@ elementi().size()==  \old(elementi().size()) + (\old(find(k)) == null ? 1 : 0)
public Elemento<V,K> insert (K k, V v) {}

//@ !elementi().contains(\result) &&
//@(\forall int i; 0\le i && i < \old(elementi().size());
//@       \old(elementi().get(i)).key() != k ==>
//@       elementi().contains(\old(elementi().get(i)))) &&
//@ elementi().size()==  \old(elementi().size()) + (\old(find(k)) == null ? 0 : -1)
public void remove (Elemento<V,K> e){}
```

b) La specifica del Dizionario viene successivamente estesa a quella di un Dizionario Ordinato, per tenere conto del caso in cui le chiavi siano ordinate totalmente. L'utilità delle chiave ordinate è ovviamente di permettere implementazioni più efficienti dei metodi insert, remove, find.

Descrivete una possibile soluzione al problema della specifica delle chiavi ordinate, motivando le vostre scelte. La vostra soluzione deve soddisfare i principi fondamentali della progettazione a oggetti. Mostrate anche quali sono i principi utilizzati, e come mai sono soddisfatti.

Sketch Soluzione Si pu specificare che le chiavi sono ordinate richiedendo ad esempio che il costruttore di DizionarioOrdinato debba ricevere come parametro un oggetto che implementa l'interfaccia Comparator$<$ $K>$.

Quindi DizionarioOrdinato puo' essere definita come la seguente estensione di Dizionario:

```
public class DizionarioOrdinato<V,K> extends Dizionario<V,K> {

//costruisce un DizionarioOrdinato vuoto usando un comparatore per le chiavi:
public DizionarioOrdinato(Comparator<K> comparatore);

//restituisce una lista di tutti e soli gli elementi di this,
//secondo l'ordine delle chiavi;
public ArrayList<Elemento<V,K>> elementi();
}
```

La Classe DizionarioOrdinato verifica il principio di sostituzione: il costruttore infatti non e' ereditato e l'unico metodo ridefinito, elementi(), rafforza la postcondizione originale. Si noti che la classe DizionarioOrdinato, quasi certamente, reimplementera' opportunamente i metodi find, insert e remove per sfruttare l'ordinamento, anche se non ne modifica (necessariamente) la specifica.

Una soluzione alternativa e' usare l'interfaccia Comparable:

Ad esempio:

```
public class DizionarioOrdinato2<V, K extends Comparable<? super K>>  extends Diziona

//costruisce un DizionarioOrdinato2 vuoto
public DizionarioOrdinato2(){}

//restituisce una lista di tutti e soli gli elementi di this, nell'ordine delle chiav
public ArrayList<Elemento<V,K>>  elementi(){return new ArrayList<Elemento<V,K>>();}
}
```

Questa classe impone un vincolo aggiuntivo su K, "suggerendo" che vi sia una qualche violazione della regola dei metodi (il vincolo su K e' una specie di precondizione). Tuttavia questo vincolo riguarda solo il costruttore (che riceve come parametro ¡K¿), quindi non comporta violazione della regola dei metodi. Anche questa soluzione verifica quindi il principio di sostituzione.

Per confrontare le due soluzioni, si consideri il seguente metodo che riceve come parametro un Dizionario in cui i valori sono String e le chiavi sono istanze di una opportuna classe Orario:

```
public static void foo(Dizionario<String,Orario> d) {...}
```

La soluzione del DizionarioOrdinato, che non impone vincoli sul parametro K, permette di istanziare e passare a foo(..) un parametro di tipo $DizionarioOrdinato < String, Orario >$, purche' per Orario sia definita una classe a parte che implementa $Comparator < Orario >$ (una cui istanza viene passata al costruttore di DizionarioOrdinato). Non e' quindi necessario modificare Orario.

La soluzione del DizionarioOrdinato2 impone invece il vincolo che il parametro K implementi l'interfaccia $Comparable < Orario >$ (o anche un $Comparable < T >$ in cui T e' sovratipo di Orario) Quindi se Orario non implementa Comparable, diventa necessario modificare la classe Orario stessa. Non solo, questa soluzione potrebbe essere impossibile qualora l'ordine che si vule usare per le chiavi non sia lo stesso ordinamento implementato da Comparable.

In linea di massima, la prima soluzione e' da preferire in quanto piu' generale: non costringe a modificare classi esistenti usate come chiavi, e permette maggiore flessibilita' nel definire l'ordinamento.

Esercizio 3

Si progetti, utilizzando un diagramma delle classi UML, la struttura di un quotidiano. Ogni giornale è composto da un certo numero di pagine; ogni pagina è organizzata in trenta slot: sei segmenti per ognuna delle cinque colonne. Il giornale contiene articoli e inserzioni pubblicitarie. Ogni articolo ha un titolo, un autore e un corpo. Ogni inserzione pubblicitaria contiene un'immagine e una dimensione. Gli articoli possono essere accompagnati da fotografie.

Articoli, fotografie e inserzioni devono essere contenuti in uno o più slot. Ogni pagina può contenere un numero ben preciso di articoli, fotografie e inserzioni; alcuni elementi potrebbero essere disposti su piú pagine. La prima pagina ha un'organizzazione leggermente diversa: la prima fila di slot contiene il nome del giornale, la data e altre informazioni accessorie.

Il giornale impaginato correttamente deve occupare tutte le pagine a disposizione. Ovvero non devono restare pagine occupate solo parzialmente o contenuti non stampati per mancanza di spazio. Dopo aver identifcato attributi e metodi necessari, si scriva anche la specifica del metodo **impaginabile** della classe **Quotidiano** per controllare se i contenuti a disposizione occupano esattamente il numero di slot della foliazione stabilita.

Esercizio 3

Si consideri il metodo seguente e si generi un insieme “minimo” di casi di test che soddisfi il criterio di copertura delle diramazioni (*branch coverage*) e che copra anche le condizioni limite.

```
00 int cerca(int[] a, int v) {
01    int r = -1, i = 0;
02     boolean t = false;
03
04     if (a == null)
05        return r;
06
07     while(i < a.length && t == false) {
08        if (a[i] == v) {
09           t = true;
10           r = i;
11        }
12        else
13           i++;
14     }
15    return r;
16  }
```